

PE-EC801C

Error Correcting Codes

Combined Module Notes

Modules 1 – 5

Semester VIII • 3L:0T:0P • 3 Credits

CGEC | B.Tech ECE (2023–27)

Biraj Sarkar

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Contents

Module 1: Introduction to Block Coding	5
1 Introduction to Block Coding	5
1.1 Vector Space Structure of Codewords	5
1.2 Weight and Distance Metrics	5
1.3 Error Detection and Correction Capabilities	6
2 Systematic Linear Block Codes	6
2.1 The Generator Matrix G	6
2.2 The Parity Check Matrix H	7
3 Optimum Decoding for the Binary Symmetric Channel	7
3.1 The Binary Symmetric Channel (BSC)	7
3.2 Maximum Likelihood (ML) Decoding	8
4 Syndrome Decoding on Symmetric Channels	8
4.1 Properties of the Syndrome	8
4.2 The Standard Array and Coset Leaders	9
4.3 Syndrome Decoding Algorithm	9
5 Hamming Codes	9
5.1 Structure of the Parity Check Matrix	10
5.2 Decoding of Hamming Codes	10
6 Weight Enumerators and McWilliams Identities	10
6.1 The Weight Enumerator Polynomial	10
6.2 The McWilliams Identities	11
7 Perfect Codes	11
7.1 The Hamming Bound (Sphere-Packing Bound)	11
7.2 Examples of Perfect Codes	12
Module 2: Algebraic Background — Groups, Rings and Fields	16
1 Algebraic Background — Groups, Rings and Fields	16
1.1 Finite Fields (Galois Fields)	16
1.2 Extension Field Construction	16
1.3 Primitive Elements and Minimal Polynomials	17
2 Factorization of $X^n - 1$ over a Finite Field	17
3 Cyclic Codes	18
3.1 Polynomial Representation	18
3.2 The Generator Polynomial $g(x)$	18
3.3 Systematic Encoding Algorithm	19
4 Feedback Shift Register Encoders and Syndrome Calculators	19
4.1 Systematic FSR Encoder Circuit	19

4.2	FSR Syndrome Calculator Circuit	20
5	Spectral Properties of Cyclic Codes	20
5.1	Spectral Constraints and Ideals	20
Module 3: BCH Codes — Definition and Construction		24
1	BCH Codes — Definition and Construction	24
1.1	Definition of Binary BCH Codes	24
1.2	Design Distance vs. Minimum Distance	24
2	Idempotents and Mattson-Solomon Polynomials	25
2.1	Idempotents	25
2.2	Mattson-Solomon Polynomials	25
3	Reed-Solomon Codes	26
3.1	Basic Parameters	26
3.2	Error and Burst Correction Capability	26
4	MDS (Maximum Distance Separable) Codes	26
4.1	Properties of MDS Codes	27
5	Justeen, Alterant and Goppa Codes	27
5.1	Justeen Codes	27
5.2	Alterant Codes	27
5.3	Goppa Codes	27
5.4	Generalized BCH Codes	28
6	Code Classes Hierarchy	28
Module 4: The BCH/RS Decoding Pipeline		32
1	The BCH/RS Decoding Pipeline	32
2	Syndrome Polynomial and the Key Equation	33
2.1	Syndrome Calculations	33
2.2	The Error Locator Polynomial	33
2.3	The Key Equation	33
3	The Berlekamp-Massey Algorithm	33
3.1	Iterative Steps of the BM Algorithm	34
4	Massey's Shift Register Synthesis	34
5	A Fast Berlekamp-Massey Algorithm	35
6	Chien Search and Forney's Algorithm	35
6.1	Chien Search	35
6.2	Forney's Algorithm	35
Module 5: Introduction to Convolutional Codes		39
1	Introduction to Convolutional Codes	39

1.1	Core Parameters	39
1.2	Mathematical Representations	39
2	Structural Representations	40
2.1	State Transition Diagram	40
2.2	Tree Diagram	41
2.3	Trellis Diagram	41
3	Distance Properties of Convolutional Codes	42
3.1	Free Distance (d_{free})	42
3.2	Catastrophic Error Propagation	42
4	The Viterbi Decoding Algorithm	43
4.1	Algorithmic Steps	43
5	Sequential Decoding Algorithms	43
5.1	Wozencraft's Algorithm	43
5.2	Fano's Algorithm (Fann's Algorithm)	44
5.3	The Stack Algorithm (Zigangirov-Jelinek)	44
6	Worked Example of Viterbi Decoding	44
6.1	Step-by-Step Trellis Computations	44
6.2	Traceback Process	46

Module 1

Introduction to Block Coding

1. Introduction to Block Coding

Error control coding is a technique used to detect and correct transmission errors introduced by noisy communication channels. In **block coding**, a stream of information bits is partitioned into blocks of size k . The channel encoder maps each k -bit message block into a larger n -bit codeword by adding $n - k$ redundant parity-check bits.

Key Coding Definitions

- **Code Word (c):** The output block of the channel encoder consisting of n bits.
- **Block Length (n):** The total number of bits in a codeword.
- **Message Length (k):** The number of actual information bits in the block.
- **Redundancy ($n - k$):** The number of check bits added for error detection/correction.
- **Code Rate (R):** The ratio of information bits to total bits, representing transmission efficiency:

$$R = \frac{k}{n}$$

1.1 Vector Space Structure of Codewords

The set of all binary n -tuples forms a vector space V_n over the binary Galois Field $\text{GF}(2) = \{0, 1\}$. There are 2^n possible vectors in V_n . A binary block code C of length n with 2^k codewords is a subset of V_n . If C is a vector subspace of V_n , it is called a **linear block code**. This algebraic structure implies that the sum (modulo-2 addition) of any two codewords in C is also a valid codeword in C :

$$\forall c_1, c_2 \in C \implies c_1 \oplus c_2 \in C$$

1.2 Weight and Distance Metrics

Error correction capability depends on the distance between codewords. We define the following metrics over the binary field:

Weight and Distance Definitions

- **Hamming Weight ($w(x)$):** The number of non-zero elements in a vector $x \in V_n$.

- **Hamming Distance** ($d(x, y)$): The number of positions in which two vectors x and y differ. This is equivalent to the weight of their modulo-2 sum:

$$d(x, y) = w(x \oplus y)$$

- **Minimum Distance** (d_{min}): The smallest Hamming distance between any two distinct codewords in the code C :

$$d_{min} = \min\{d(c_i, c_j) \mid c_i, c_j \in C, c_i \neq c_j\}$$

For any linear block code, the minimum distance is equal to the minimum weight of any non-zero codeword in the code:

$$d_{min} = \min\{w(c) \mid c \in C, c \neq 0\}$$

1.3 Error Detection and Correction Capabilities

Error Bounds

Let d_{min} be the minimum distance of a block code C .

1. **Error Detection:** A code can detect up to t_d random errors per codeword if:

$$d_{min} \geq t_d + 1$$

2. **Error Correction:** A code can correct up to t_c random errors per codeword if:

$$d_{min} \geq 2t_c + 1 \implies t_c = \left\lfloor \frac{d_{min} - 1}{2} \right\rfloor$$

2. Systematic Linear Block Codes

A block code is **systematic** if the message bits appear unaltered at the beginning (or end) of the codeword, followed by the parity-check bits. A systematic codeword vector c can be partitioned as:

$$c = [m_0, m_1, \dots, m_{k-1}, p_0, p_1, \dots, p_{n-k-1}] = [m \mid p]$$

2.1 The Generator Matrix G

The encoding process is defined by a $k \times n$ matrix called the **Generator Matrix** G . Every codeword is generated by multiplying the k -bit message vector m by G :

$$c = m \cdot G$$

For a systematic linear block code, G is structured as:

$$G = [I_k \mid P]$$

where I_k is the $k \times k$ identity matrix (generating the message part) and P is a $k \times (n - k)$ parity-check coefficient matrix (generating the redundant part).

$$G = \begin{bmatrix} 1 & 0 & \dots & 0 & p_{0,0} & p_{0,1} & \dots & p_{0,n-k-1} \\ 0 & 1 & \dots & 0 & p_{1,0} & p_{1,1} & \dots & p_{1,n-k-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 & p_{k-1,0} & p_{k-1,1} & \dots & p_{k-1,n-k-1} \end{bmatrix}$$

The parity bits are computed by:

$$p_j = \sum_{i=0}^{k-1} m_i p_{i,j} \pmod{2}$$

2.2 The Parity Check Matrix H

The parity check matrix H is an $(n - k) \times n$ matrix used to verify if a received vector is a valid codeword. By definition, any valid codeword $c \in C$ must satisfy the orthogonal relation:

$$c \cdot H^T = \mathbf{0}$$

For a systematic code where $G = [I_k \mid P]$, the parity check matrix is:

$$H = [P^T \mid I_{n-k}]$$

where P^T is the transpose of the parity matrix and I_{n-k} is the $(n - k) \times (n - k)$ identity matrix.

Verification of Orthogonality

Let us verify the relationship $G \cdot H^T = \mathbf{0}$:

$$G \cdot H^T = [I_k \mid P] \begin{bmatrix} P \\ I_{n-k} \end{bmatrix} = I_k \cdot P \oplus P \cdot I_{n-k} = P \oplus P = \mathbf{0} \pmod{2}$$

Since $c = m \cdot G$, we have:

$$c \cdot H^T = (m \cdot G) \cdot H^T = m \cdot (G \cdot H^T) = m \cdot \mathbf{0} = \mathbf{0}$$

This confirms that the parity check matrix successfully screens for codewords.

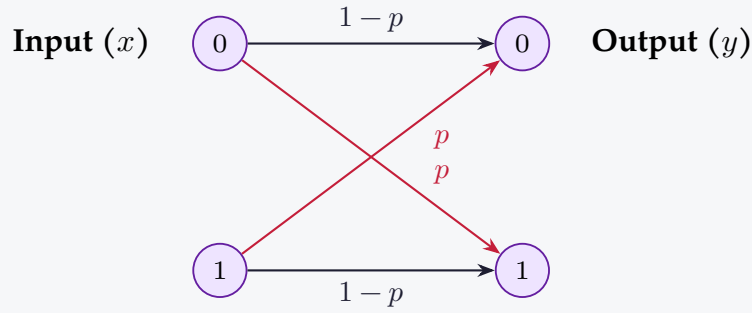
3. Optimum Decoding for the Binary Symmetric Channel

Let c be the transmitted codeword and $r = [r_0, r_1, \dots, r_{n-1}]$ be the received vector. Due to channel noise, $r = c \oplus e$, where $e = [e_0, e_1, \dots, e_{n-1}]$ is the binary error vector ($e_i = 1$ indicates a bit inversion error at position i).

3.1 The Binary Symmetric Channel (BSC)

The BSC is a discrete memoryless channel with binary input, binary output, and a crossover probability p ($p < 0.5$).

Binary Symmetric Channel Model



3.2 Maximum Likelihood (ML) Decoding

An optimum decoder minimizes the probability of decoding error. The maximum likelihood decoding rule selects the codeword $c^{(i)}$ that maximizes the conditional probability $P(r | c^{(i)})$:

$$P(r | c^{(i)}) = p^{d(r, c^{(i)})} (1 - p)^{n - d(r, c^{(i)})}$$

Since $p < 0.5 \implies 1 - p > p$, we can rewrite this by taking the logarithm:

$$\ln P(r | c^{(i)}) = d(r, c^{(i)}) \ln \left(\frac{p}{1 - p} \right) + n \ln(1 - p)$$

Since $\ln(p/(1 - p)) < 0$ for $p < 0.5$, maximizing $P(r | c^{(i)})$ is mathematically equivalent to minimizing the Hamming distance:

ML Decoding Rule: Choose $c \in C$ that minimizes $d(r, c)$

Thus, over a BSC, Maximum Likelihood decoding simplifies to **minimum distance decoding**.

4. Syndrome Decoding on Symmetric Channels

Instead of calculating the distance from r to all 2^k codewords, we can use the parity check matrix to compute a **Syndrome** vector S .

The Syndrome

The syndrome of a received vector r is defined as:

$$S = r \cdot H^T$$

4.1 Properties of the Syndrome

1. $S = \mathbf{0}$ if and only if r is a valid codeword.
2. The syndrome depends **only on the error pattern** e , not on the transmitted codeword c :

$$S = r \cdot H^T = (c \oplus e) \cdot H^T = (c \cdot H^T) \oplus (e \cdot H^T) = \mathbf{0} \oplus (e \cdot H^T) = e \cdot H^T$$

3. Two received vectors r_1 and r_2 have the same syndrome if and only if they differ by a valid codeword.

4.2 The Standard Array and Coset Leaders

A standard array partitions the 2^n possible received vectors into a table of 2^{n-k} rows (called **cosets**) and 2^k columns.

Standard Array Construction Algorithm:

1. Place all 2^k valid codewords in the first row, starting with the all-zero codeword $c_0 = \mathbf{0}$ on the far left.
2. From the remaining unused vectors in V_n , choose a vector e_1 of minimum weight. Place it in the first column of the second row. This vector is the **coset leader**.
3. Fill the second row by adding the coset leader e_1 to each codeword: $e_1 \oplus c_i$.
4. Repeat this process: choose a minimum weight unused vector e_j as the next coset leader, and fill the row with $e_j \oplus c_i$, until all 2^n vectors are placed.

Standard Array Layout

Coset Leader (e_j)	Codeword $c_0 = \mathbf{0}$	Codeword c_1	...	Codeword c_{2^k-1}
$e_0 = \mathbf{0}$	$\mathbf{0}$	c_1	...	c_{2^k-1}
e_1	e_1	$e_1 \oplus c_1$	...	$e_1 \oplus c_{2^k-1}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$e_{2^{n-k}-1}$	$e_{2^{n-k}-1}$	$e_{2^{n-k}-1} \oplus c_1$	...	$e_{2^{n-k}-1} \oplus c_{2^k-1}$

Each row (coset) corresponds to a unique syndrome. The coset leader is the **most likely error pattern** for that syndrome because it has the minimum weight in its coset.

4.3 Syndrome Decoding Algorithm

Syndrome Decoding Steps

Step 1: Compute the syndrome of the received vector: $S = r \cdot H^T$.

Step 2: Find the coset leader e that matches this syndrome: $e \cdot H^T = S$.

Step 3: Correct the received vector to obtain the decoded codeword:

$$\hat{c} = r \oplus e$$

5. Hamming Codes

Hamming codes are a class of single-error-correcting linear block codes. For any integer $m \geq 2$, there exists a Hamming code with the following parameters:

Hamming Code Parameters

- **Parity bits:** $n - k = m$
- **Code length:** $n = 2^m - 1$
- **Message length:** $k = 2^m - 1 - m$
- **Minimum distance:** $d_{min} = 3$
- **Error Correction:** Corrects all single errors ($t_c = 1$).

5.1 Structure of the Parity Check Matrix

The parity check matrix H of a Hamming code has m rows and $2^m - 1$ columns. The columns of H consist of all possible non-zero binary m -tuples, arranged in any order. In a systematic Hamming code:

$$H = [P^T \mid I_m]$$

where the columns of P^T consist of all non-zero binary m -tuples with weight ≥ 2 .

5.2 Decoding of Hamming Codes

When a single error occurs at position i , the received vector is $r = c \oplus e_i$, where e_i has a 1 at index i and zeros elsewhere. The syndrome is:

$$S = r \cdot H^T = (c \oplus e_i) \cdot H^T = e_i \cdot H^T = h_i^T$$

where h_i^T is the transpose of the i -th column of H . Therefore, the syndrome S is **identical to the column of H corresponding to the error location**. If the columns of H are arranged in order of their decimal equivalents, S directly represents the index of the corrupted bit in binary.

6. Weight Enumerators and McWilliams Identities

The performance of a block code is determined by its **Weight Distribution**. Let A_i be the number of codewords in C with Hamming weight i . The set $\{A_0, A_1, \dots, A_n\}$ is the weight spectrum of the code.

6.1 The Weight Enumerator Polynomial

The weight distribution can be expressed as a polynomial:

Weight Enumerator Polynomial

$$A(z) = \sum_{i=0}^n A_i z^i = A_0 + A_1 z + A_2 z^2 + \dots + A_n z^n$$

where $A_0 = 1$ (the all-zero codeword).

6.2 The McWilliams Identities

The McWilliams identities relate the weight enumerator polynomial $A(z)$ of a linear block code C of length n to the weight enumerator polynomial $A^\perp(z)$ of its **dual code** $C^\perp$. The dual code $C^\perp$ is the set of all vectors orthogonal to every codeword in C , having dimension $n - k$ and generator matrix H .

McWilliams Identity Theorem

For a binary $[n, k]$ linear code C with weight enumerator $A(z)$ and dual code $C^\perp$ with weight enumerator $A^\perp(z)$:

$$A^\perp(z) = \frac{1}{2^k} (1+z)^n A\left(\frac{1-z}{1+z}\right)$$

Alternatively, writing as a homogeneous polynomial in two variables x and y :

$$W_{C^\perp}(x, y) = \frac{1}{2^k} W_C(x + y, x - y)$$

This identity is useful because it allows us to find the weight distribution of a code with large k by analyzing its dual code which has a smaller dimension $n - k$ (e.g., finding the weight distribution of a (127, 120) Hamming code by analyzing its (127, 7) dual code).

7. Perfect Codes

For any block code correcting up to t errors, the sphere of radius t centered around each codeword must be disjoint. The total volume of these spheres cannot exceed the size of the vector space 2^n .

7.1 The Hamming Bound (Sphere-Packing Bound)

The number of vectors in a sphere of radius t in V_n is:

$$V(n, t) = \sum_{i=0}^t \binom{n}{i}$$

For 2^k codewords, we must have:

$$2^k \sum_{i=0}^t \binom{n}{i} \leq 2^n \implies \sum_{i=0}^t \binom{n}{i} \leq 2^{n-k}$$

Perfect Code Definition

A block code is a **Perfect Code** if it satisfies the Hamming bound with equality:

$$2^k \sum_{i=0}^t \binom{n}{i} = 2^n$$

In a perfect code, every vector in the space V_n lies in exactly one sphere of radius t around a codeword. There are no "wasted" vectors that cannot be corrected.

7.2 Examples of Perfect Codes

Only a few families of perfect codes exist:

1. **Single-Error-Correcting Hamming Codes** ($t = 1$): Satisfy: $1 + n = 2^{n-k} \implies 1 + (2^m - 1) = 2^m \quad \checkmark$

2. **The Binary Golay Code** ($[23, 12, 7]$): Corrects $t = 3$ errors.

$$\sum_{i=0}^3 \binom{23}{i} = 1 + 23 + 253 + 1771 = 2048 = 2^{11} = 2^{23-12} \quad \checkmark$$

3. **The Ternary Golay Code** ($[11, 6, 5]$): Defined over $\text{GF}(3)$, corrects $t = 2$ errors.

$$\sum_{i=0}^2 \binom{11}{i} 2^i = 1 + 11(2) + 55(4) = 1 + 22 + 220 = 243 = 3^5 = 3^{11-6} \quad \checkmark$$

4. **Trivial Perfect Codes**: The whole space ($d_{\min} = 1$), and binary repetition codes of odd length n ($d_{\min} = n, t = (n - 1)/2$).

Quick Revision — Module 1

Linear Block Codes | PE-EC801C

Key Formulas and Metrics at a Glance

Code Rate	$R = k/n$
Hamming Distance	$d(x, y) = w(x \oplus y)$
Error Correction Limit	$t_c = \lfloor (d_{min} - 1)/2 \rfloor$
Systematic Generator Matrix	$G = [I_k \mid P]$
Parity Check Matrix	$H = [P^T \mid I_{n-k}]$
Orthogonality Condition	$G \cdot H^T = \mathbf{0} \pmod{2}$ and $c \cdot H^T = \mathbf{0}$
Syndrome Vector	$S = r \cdot H^T = e \cdot H^T$
Hamming Bound (Sphere-Packing)	$2^k \sum_{i=0}^t \binom{n}{i} \leq 2^n$
McWilliams Identity	$A^\perp(z) = \frac{1}{2^k} (1+z)^n A\left(\frac{1-z}{1+z}\right)$
Hamming Code Parameters	Length $n = 2^m - 1$, Info bits $k = 2^m - 1 - m$, $d_{min} = 3$

One-Line Definitions

- **Linear Block Code:** A code where the modulo-2 sum of any two codewords is also a valid codeword.
- **Minimum Distance (d_{min}):** The smallest Hamming distance between any two distinct codewords in a code.
- **Syndrome:** A vector calculated as $r \cdot H^T$ that indicates if errors occurred and helps locate them.
- **Coset Leader:** The vector with the smallest Hamming weight in a coset, representing the most likely error.
- **Hamming Code:** A class of perfect single-error-correcting codes with minimum distance $d_{min} = 3$.
- **Perfect Code:** A code that satisfies the Hamming (sphere-packing) bound with strict equality.
- **Dual Code ($C^\perp$):** The set of all vectors orthogonal to every codeword in the original code C .
- **Binary Symmetric Channel:** A binary input/output channel where bit-flip probability is equal for 0 and 1.

Frequently Asked / Viva Questions

Q1. What is the difference between systematic and non-systematic codes?

In systematic codes, the message bits are placed directly at the beginning or end of the codeword, making extraction trivial. In non-systematic codes, message and parity bits are interleaved or mathematically combined throughout the codeword.

Q2. Prove that $d_{min} = w_{min}$ for linear block codes.

For linear block codes, if $c_1, c_2 \in C$ are codewords, then $c_1 \oplus c_2 = c_3 \in C$ is also a codeword. The distance is $d(c_1, c_2) = w(c_1 \oplus c_2) = w(c_3)$. Thus, minimizing distance between distinct codewords is equivalent to finding the minimum weight of non-zero codewords.

Q3. What is the physical significance of the syndrome vector?

The syndrome represents the parity-check violations of the received vector. Because $S = e \cdot H^T$, it depends strictly on the channel error pattern, not on the transmitted data, allowing isolated error isolation.

Q4. Explain the ML decoding criteria over a Binary Symmetric Channel.

Maximum Likelihood decoding selects the codeword that maximizes $P(r | c)$. On a BSC with crossover probability $p < 0.5$, this probability increases as the distance $d(r, c)$ decreases. Thus, ML decoding reduces to minimum Hamming distance decoding.

Q5. Why are Hamming codes considered perfect codes?

Hamming codes have $d_{min} = 3$, meaning they correct $t = 1$ error. They satisfy the sphere-packing bound with equality since $\sum_{i=0}^1 \binom{n}{i} = 1 + n = 1 + 2^m - 1 = 2^m = 2^{n-k}$, leaving no unused vectors.

Q6. What is the role of a standard array in syndrome decoding?

A standard array groups all possible 2^n received vectors into 2^{n-k} cosets. Each coset shares the same syndrome, and the coset leader (first vector of the row) is chosen as the minimum-weight error pattern for correction.

Q7. State the McWilliams identities and explain their utility.

The McWilliams identities relate the weight enumerator polynomial of a linear code to that of its dual code: $A^\perp(z) = 2^{-k}(1+z)^n A((1-z)/(1+z))$. This allows for the simplified calculation of the weight spectrum of high-rate codes (large k) by analyzing their lower-dimension dual codes.

Q8. What is the Golay code, and why is it important?

The binary Golay code is a $[23, 12, 7]$ perfect code that corrects all patterns of up to 3 random errors. It is the only known binary perfect code that corrects multiple errors ($t > 1$) besides repetition codes.

Q9. Can a linear block code correct more errors than its minimum distance implies?

No. A code can guarantee the correction of all error patterns of weight up to $t = \lfloor (d_{min} - 1)/2 \rfloor$. While it may correct some specific patterns of weight $> t$, it cannot correct all of them.

Frequently Asked / Viva Questions (contd.)

Q10. What happens if the syndrome matches no single column in a Hamming parity check matrix?

In a Hamming code, if the syndrome is non-zero but doesn't match a column representing a single-bit error (which is impossible since columns represent all non-zero m -tuples), it indicates that more than one error has occurred, leading to decoding failure.

Comparison of Perfect and Near-Perfect Block Codes

Code Type	Parameters $[n, k, d_{min}]$	Error Correction (t)	Perfect Status
Hamming Codes	$[2^m - 1, 2^m - 1 - m, 3]$	$t = 1$	Perfect
Binary Golay Code	$[23, 12, 7]$	$t = 3$	Perfect
Ternary Golay Code	$[11, 6, 5]$	$t = 2$	Perfect
Repetition Codes	$[n, 1, n]$ (n odd)	$t = (n - 1)/2$	Perfect (Trivial)

Module 2

Algebraic Background — Groups, Rings and Fields

1. Algebraic Background — Groups, Rings and Fields

To design and analyze cyclic, BCH, and Reed-Solomon codes, we must establish their underlying algebraic structures.

Algebraic Structures Summary

- **Group:** A set G with a binary operation $*$ satisfying: closure ($a * b \in G$), associativity ($a * (b * c) = (a * b) * c$), existence of identity e ($a * e = a$), and existence of inverse a^{-1} ($a * a^{-1} = e$). If $a * b = b * a$, it is an **Abelian Group**.
- **Ring:** A set R with two binary operations (addition $+$ and multiplication $\cdot$) where $(R, +)$ is an Abelian group, multiplication is associative and distributive over addition, and contains a multiplicative identity 1.
- **Field:** A ring F where multiplication is commutative, and every non-zero element $a \in F$ has a multiplicative inverse a^{-1} such that $a \cdot a^{-1} = 1$.

1.1 Finite Fields (Galois Fields)

A field containing a finite number of elements is a **Galois Field**, denoted as $\text{GF}(q)$. A finite field exists if and only if q is a prime power: $q = p^m$, where p is a prime number (called the **characteristic** of the field) and m is a positive integer.

- For $m = 1$, $\text{GF}(p)$ is the prime field consisting of integers $\{0, 1, \dots, p-1\}$ under modulo- p arithmetic.
- For $m > 1$, $\text{GF}(2^m)$ is an extension field constructed using polynomials.

1.2 Extension Field Construction

To construct $\text{GF}(2^m)$, we choose an **irreducible polynomial** $p(x)$ (a polynomial that cannot be factored into lower-degree polynomials over $\text{GF}(2)$) of degree m . Let α be a root of $p(x)$, such that $p(\alpha) = 0$. The elements of $\text{GF}(2^m)$ can be represented in three ways: 1. **Power representation:** $\{0, \alpha^0, \alpha^1, \dots, \alpha^{2^m-2}\}$. 2. **Polynomial representation:** Binary combinations of $\{1, \alpha, \alpha^2, \dots, \alpha^{m-1}\}$. 3. **Vector representation:** Binary coefficients of the polynomial representation.

Construction of $\text{GF}(2^3)$ using $p(x) = x^3 + x + 1$

Let α be a root of $p(x) \Rightarrow \alpha^3 \oplus \alpha \oplus 1 = 0 \Rightarrow \alpha^3 = \alpha \oplus 1$. The $2^3 = 8$ elements of $\text{GF}(8)$ are:

Power Form	Polynomial Form	Vector Form (a_2, a_1, a_0)
0	0	(0, 0, 0)
α^0	1	(0, 0, 1)
α^1	α	(0, 1, 0)
α^2	α^2	(1, 0, 0)
α^3	$\alpha \oplus 1$	(0, 1, 1)
α^4	$\alpha^2 \oplus \alpha$	(1, 1, 0)
α^5	$\alpha^2 \oplus \alpha \oplus 1$	(1, 1, 1)
α^6	$\alpha^2 \oplus 1$	(1, 0, 1)

1.3 Primitive Elements and Minimal Polynomials

- **Primitive Element:** An element $\beta \in \text{GF}(2^m)$ whose successive powers generate all $2^m - 1$ non-zero elements of the field.
- **Conjugate Elements:** If $\beta \in \text{GF}(2^m)$, its conjugates over $\text{GF}(2)$ are:

$$\beta, \beta^2, \beta^{2^2}, \dots, \beta^{2^{d-1}}$$

where d is the smallest integer such that $\beta^{2^d} = \beta$.

- **Minimal Polynomial $(m(x))$:** The lowest-degree polynomial with binary coefficients that has β (and its conjugates) as a root:

$$m(x) = \prod_{i=0}^{d-1} (x - \beta^{2^i})$$

2. Factorization of $X^n - 1$ over a Finite Field

Let n be the block length of a cyclic code. The polynomial $X^n \oplus 1$ can be factored into a product of irreducible polynomials over $\text{GF}(2)$:

$$X^n \oplus 1 = f_1(x)f_2(x) \dots f_r(x)$$

The roots of these irreducible factors are elements of $\text{GF}(2^m)$, where m is the multiplicative order of 2 modulo n . Every irreducible factor $f_i(x)$ is the minimal polynomial of a root of $X^n \oplus 1$.

Factorization Example for $n = 7$

We wish to factor $X^7 \oplus 1$ over $\text{GF}(2)$. The roots of $X^7 \oplus 1 = 0$ are the 7-th roots of unity in $\text{GF}(8)$, which are $\{1, \alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6\}$. Grouping roots into conjugate classes (cyclotomic cosets):

- Coset 1: $\{1\}$ (minimal polynomial: $x \oplus 1$)
- Coset 2: $\{\alpha, \alpha^2, \alpha^4\}$ (minimal polynomial: $x^3 \oplus x \oplus 1$)
- Coset 3: $\{\alpha^3, \alpha^6, \alpha^5\}$ (minimal polynomial: $x^3 \oplus x^2 \oplus 1$)

Therefore, the factorization of $X^7 \oplus 1$ over $\text{GF}(2)$ is:

$$X^7 \oplus 1 = (x \oplus 1)(x^3 \oplus x \oplus 1)(x^3 \oplus x^2 \oplus 1)$$

3. Cyclic Codes

A **cyclic code** is a linear block code where any cyclic shift of a codeword is also a valid codeword.

Formal Cyclic Shift Property

If $c = (c_0, c_1, \dots, c_{n-1}) \in C$ is a codeword, then its cyclic shift:

$$c^{(1)} = (c_{n-1}, c_0, c_1, \dots, c_{n-2}) \in C$$

is also a valid codeword.

3.1 Polynomial Representation

Codewords are represented as polynomials of degree at most $n - 1$:

$$c(x) = c_0 \oplus c_1x \oplus c_2x^2 \oplus \dots \oplus c_{n-1}x^{n-1}$$

A cyclic shift corresponds to multiplying $c(x)$ by x modulo $X^n \oplus 1$:

$$x \cdot c(x) \pmod{X^n \oplus 1} = c_{n-1} \oplus c_0x \oplus c_1x^2 \oplus \dots \oplus c_{n-2}x^{n-1}$$

Algebraically, a cyclic code of length n is an **ideal** in the quotient polynomial ring $\text{GF}(2)[x]/(X^n \oplus 1)$.

3.2 The Generator Polynomial $g(x)$

A cyclic code is uniquely defined by a single generator polynomial $g(x)$.

Properties of the Generator Polynomial

- $g(x)$ is the unique, monic polynomial of minimum degree in the code.
- The degree of $g(x)$ is equal to the number of parity check bits: $\deg(g(x)) = n - k$.
- $g(x)$ is a factor of $X^n \oplus 1$:

$$X^n \oplus 1 = g(x) \cdot h(x)$$

where $h(x)$ is the **parity check polynomial** of degree k .

Every codeword polynomial $c(x)$ in the code can be written as a product of a message polynomial $m(x)$ (of degree $< k$) and the generator polynomial $g(x)$:

$$c(x) = m(x) \cdot g(x)$$

3.3 Systematic Encoding Algorithm

To construct a systematic cyclic code where the message bits occupy the higher-order coefficients of $c(x)$, we use the following steps:

Systematic Cyclic Encoding Steps

Step 1: Multiply the message polynomial $m(x)$ by x^{n-k} to shift it into the systematic position.

Step 2: Divide $x^{n-k}m(x)$ by the generator polynomial $g(x)$ to find the remainder $d(x)$:

$$x^{n-k}m(x) = q(x)g(x) \oplus d(x)$$

where $\deg(d(x)) < n - k$.

Step 3: Construct the codeword polynomial:

$$c(x) = x^{n-k}m(x) \oplus d(x) = q(x)g(x)$$

Since addition and subtraction are identical in binary (modulo-2), adding $d(x)$ to $x^{n-k}m(x)$ cancels the remainder, leaving a polynomial that is a multiple of $g(x)$ (hence a valid codeword).

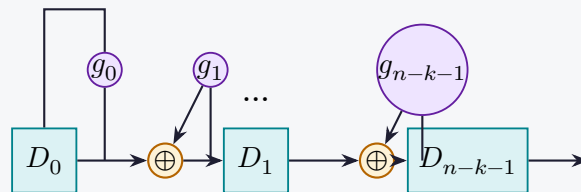
4. Feedback Shift Register Encoders and Syndrome Calculators

The polynomial division needed for systematic cyclic encoding is implemented in hardware using a **Feedback Shift Register (FSR)**.

4.1 Systematic FSR Encoder Circuit

Let $g(x) = g_0 \oplus g_1x \oplus \dots \oplus g_{n-k-1}x^{n-k-1} \oplus x^{n-k}$ be the generator polynomial. The shift register consists of $n - k$ stages.

Systematic Cyclic Encoder Circuit



The division operates as follows:

1. The gate switch is closed. The k message bits are fed into the channel and the feedback path simultaneously.
2. After k clock cycles, the shift registers contain the remainder coefficients (parity bits).
3. The gate switch is opened, disabling feedback, and the parity bits are shifted out to complete the n -bit systematic codeword.

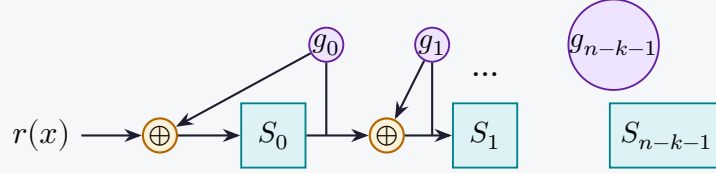
4.2 FSR Syndrome Calculator Circuit

The syndrome $S(x)$ of a received polynomial $r(x)$ is the remainder of $r(x)$ divided by $g(x)$:

$$S(x) = r(x) \pmod{g(x)}$$

This is implemented using a feedback shift register similar to the encoder.

Syndrome Calculator Circuit



5. Spectral Properties of Cyclic Codes

Like time-domain signals, codewords of cyclic codes have equivalent frequency-domain descriptions using the **Discrete Fourier Transform (DFT)** over finite fields. Let $v = (v_0, v_1, \dots, v_{n-1})$ be a vector over $\text{GF}(2^m)$ where n divides $2^m - 1$. Let α be a primitive n -th root of unity in $\text{GF}(2^m)$.

Finite Field Discrete Fourier Transform

The DFT of v , denoted $V = (V_0, V_1, \dots, V_{n-1})$, consists of elements in $\text{GF}(2^m)$ computed by:

$$V_j = \sum_{i=0}^{n-1} v_i \alpha^{ij}, \quad j = 0, 1, \dots, n-1$$

The inverse DFT is:

$$v_i = \sum_{j=0}^{n-1} V_j \alpha^{-ij}, \quad i = 0, 1, \dots, n-1$$

5.1 Spectral Constraints and Ideals

The cyclic shift of a vector v in the time domain corresponds to multiplying its spectral components by powers of α :

$$v^{(1)} \longleftrightarrow (\alpha^0 V_0, \alpha^1 V_1, \dots, \alpha^{n-1} V_{n-1})$$

Therefore, a cyclic code can be defined in the spectral domain by forcing certain spectral components to zero:

$$C = \{c \in \text{GF}(2)^n \mid C_j = 0 \text{ for all } j \in \mathcal{K}\}$$

where $\mathcal{K}$ is a set of spectral indices. This spectral property is the foundation for designing BCH and Reed-Solomon codes.

Quick Revision — Module 2

Galois Fields & Cyclic Codes | PE-EC801C

Key Formulas and Metrics at a Glance

Galois Field order	$q = p^m$ (p is prime characteristic)
Roots of Unity factor	$X^n \oplus 1 = g(x) \cdot h(x)$
Generator degree	$\deg(g(x)) = n - k$
Systematic Codeword polynomial	$c(x) = x^{n-k}m(x) \oplus [x^{n-k}m(x) \pmod{g(x)}]$
Modulo-2 addition relation	$a(x) \oplus b(x) = a(x) - b(x)$
Parity relation	$g(x) \cdot h(x) \equiv 0 \pmod{X^n \oplus 1}$
Finite Field DFT spectrum	$V_j = \sum_{i=0}^{n-1} v_i \alpha^{ij}$
Conjugate elements of β	$\beta^{2^0}, \beta^{2^1}, \beta^{2^2}, \dots, \beta^{2^{d-1}}$
Generator matrix polynomial	$G = \begin{bmatrix} g(x) \\ xg(x) \\ \vdots \\ x^{k-1}g(x) \end{bmatrix}$ in non-systematic form

One-Line Definitions

- **Abelian Group:** A group where the binary operation is commutative ($a * b = b * a$).
- **Field:** An algebraic ring where multiplication is commutative and every non-zero element has a multiplicative inverse.
- **Galois Field:** A finite field containing a prime-power number of elements, denoted $\text{GF}(p^m)$.
- **Irreducible Polynomial:** A polynomial that cannot be factored into polynomials of lower degree over the same field.
- **Primitive Element:** An element whose powers generate all non-zero elements of a finite field.
- **Minimal Polynomial:** The lowest-degree binary polynomial having a field element (and its conjugates) as roots.
- **Cyclic Code:** A linear code where a cyclic shift of any codeword produces another valid codeword.
- **Generator Polynomial ($g(x)$):** The unique lowest-degree polynomial of a cyclic code, which divides $X^n \oplus 1$.

Frequently Asked / Viva Questions

Q1. What are the differences between groups, rings, and fields?

A group has one operation (+). A ring has two operations (+ and $\cdot$) and is an Abelian group under addition. A field is a commutative ring with a multiplicative identity where every non-zero element has a unique multiplicative inverse.

Q2. Why do Galois Fields only exist for prime-power sizes p^m ?

For fields of size q , the characteristic must be a prime p . The field forms a vector space over its prime subfield $\text{GF}(p)$ of dimension m . Therefore, the number of elements in the field is $p \times p \times \cdots \times p = p^m$.

Q3. How is systematic cyclic encoding performed algebraically?

First, the message $m(x)$ is multiplied by x^{n-k} to shift it. Then, we find the remainder $d(x)$ of $x^{n-k}m(x)$ divided by $g(x)$. The systematic codeword polynomial is constructed as $c(x) = x^{n-k}m(x) \oplus d(x)$.

Q4. Why must the generator polynomial $g(x)$ divide $X^n \oplus 1$?

A cyclic code of length n is an ideal in the polynomial ring modulo $X^n \oplus 1$. The ideal is generated by $g(x)$. Since the identity codeword $0 \equiv X^n \oplus 1 \pmod{X^n \oplus 1}$ must be a multiple of $g(x)$, $g(x)$ must divide $X^n \oplus 1$.

Q5. Explain how a feedback shift register circuit performs polynomial division.

FSRs use shift register stages as memory and XOR gates as modulo-2 adders. Feedback paths multiply register states by coefficients g_i . Feeding coefficients through the loop matches polynomial long division.

Q6. What are conjugate elements in a finite field?

For an element $\beta \in \text{GF}(2^m)$, its conjugates over $\text{GF}(2)$ are its repeated squarings: $\{\beta, \beta^2, \beta^4, \dots, \beta^{2^{d-1}}\}$, where d is the cycle length. They share the same minimal polynomial.

Q7. What is the parity check polynomial $h(x)$ of a cyclic code?

The parity check polynomial is defined by $h(x) = (X^n \oplus 1)/g(x)$. It has degree k and satisfies $c(x) \cdot h(x) \equiv 0 \pmod{X^n \oplus 1}$ for any codeword $c(x)$.

Q8. What are the advantages of cyclic codes over general linear block codes?

Cyclic codes have rich algebraic properties allowing systematic design (like BCH codes). In hardware, encoding and syndrome calculations are implemented using simple feedback shift registers.

Q9. What is a primitive polynomial?

A primitive polynomial of degree m is an irreducible polynomial over $\text{GF}(2)$ that has a primitive element of $\text{GF}(2^m)$ as a root. Its roots generate the entire multiplicative group.

Q10. Explain the spectral definition of cyclic codes.

Using the DFT over finite fields, cyclic shifts correspond to multiplying spectral elements by powers of α . A cyclic code is defined by forcing specific spectral elements of the codewords to zero.

Comparison of Linear Block Codes and Cyclic Codes

Property	Linear Block Codes	Cyclic Codes
Mathematical Model	Vector Spaces over $\text{GF}(2)$	Ideals in the ring $\text{GF}(2)[x]/(X^n \oplus 1)$
Defining Matrices	Generator G and Parity H	Generator $g(x)$ and Parity $h(x)$ polynomials
Encoding Hardware	Matrix multiplication circuits	Feedback Shift Registers (FSR)
Error Correction	Systematic bounding	Rich algebraic code design (BCH/RS)

Module 3

BCH Codes — Definition and Construction

1. BCH Codes — Definition and Construction

Bose-Chaudhuri-Hocquenghem (**BCH**) codes are a powerful class of multiple-error-correcting cyclic codes. They allow for the precise design of codes that can correct a specific number of random errors t .

1.1 Definition of Binary BCH Codes

Let α be a primitive element of $\text{GF}(2^m)$, and let $n = 2^m - 1$. A t -error correcting binary BCH code of length n is defined as a cyclic code whose generator polynomial $g(x)$ has the following $2t$ consecutive powers of α as roots:

$$\alpha^1, \alpha^2, \alpha^3, \dots, \alpha^{2t}$$

Therefore, the generator polynomial is the Least Common Multiple (**LCM**) of the minimal polynomials $m_i(x)$ of these roots:

BCH Generator Polynomial

$$g(x) = \text{lcm}\{m_1(x), m_2(x), \dots, m_{2t}(x)\}$$

where $m_i(x)$ is the minimal polynomial of α^i over $\text{GF}(2)$.

1.2 Design Distance vs. Minimum Distance

Because the generator polynomial has $2t$ consecutive powers of α as roots, the minimum distance $d_{\min}$ of the BCH code satisfies the **BCH Bound**:

$$d_{\min} \geq 2t + 1$$

The value $d_d = 2t + 1$ is called the **design distance** of the BCH code. In many cases, the actual minimum distance is exactly equal to the design distance, though it can sometimes exceed it.

Design of a 2-Error Correcting BCH Code of Length $n = 15$

We wish to design a $t = 2$ error-correcting BCH code of length $n = 15 = 2^4 - 1 \Rightarrow m = 4$. The generator polynomial must have roots $\alpha^1, \alpha^2, \alpha^3, \alpha^4$. We identify the minimal polynomials of these roots in $\text{GF}(16)$:

- Root $\alpha^1, \alpha^2, \alpha^4$: conjugate class $\{\alpha, \alpha^2, \alpha^4, \alpha^8\}$ with minimal polynomial $m_1(x) = x^4 \oplus x \oplus 1$.
- Root α^3 : conjugate class $\{\alpha^3, \alpha^6, \alpha^{12}, \alpha^9\}$ with minimal polynomial $m_3(x) = x^4 \oplus x^3 \oplus x^2 \oplus x \oplus 1$.

Therefore, the generator polynomial is:

$$g(x) = \text{lcm}\{m_1(x), m_3(x)\} = m_1(x) \cdot m_3(x)$$

$$g(x) = (x^4 \oplus x \oplus 1)(x^4 \oplus x^3 \oplus x^2 \oplus x \oplus 1) = x^8 \oplus x^7 \oplus x^6 \oplus x^4 \oplus 1$$

This is a (15, 7) cyclic code. Since $\deg(g(x)) = 8$, the number of check bits is 8, leaving $k = 7$ message bits. Its minimum distance satisfies $d_{\min} \geq 2(2) + 1 = 5$.

2. Idempotents and Mattson-Solomon Polynomials

2.1 Idempotents

In the algebra of cyclic codes (the ring $\text{GF}(2)[x]/(X^n \oplus 1)$), an **idempotent** is a polynomial $e(x)$ that is equal to its own square:

$$e(x)^2 \equiv e(x) \pmod{X^n \oplus 1}$$

Every ideal (cyclic code) has a unique **generating idempotent** $e(x)$ which acts as the multiplicative identity of the code. The cyclic code can be defined directly by $e(x)$ rather than $g(x)$.

2.2 Mattson-Solomon Polynomials

The Mattson-Solomon polynomial represents a vector $v = (v_0, v_1, \dots, v_{n-1})$ of length n (where $\gcd(n, p) = 1$) as a polynomial $g_v(z)$ over the extension field $\text{GF}(p^m)$:

Mattson-Solomon Polynomial Definition

$$g_v(z) = \sum_{j=1}^n V_j z^{n-j}$$

where V_j are the finite-field DFT coefficients of v :

$$V_j = \sum_{i=0}^{n-1} v_i \alpha^{ij}$$

The coefficients of $g_v(z)$ represent the spectral properties of the code, providing an elegant way to translate time-domain constraints into algebraic conditions.

3. Reed-Solomon Codes

Reed-Solomon (RS) codes are a non-binary subclass of BCH codes. Unlike binary codes, the elements of Reed-Solomon codewords are symbols from the Galois Field $GF(2^m)$ rather than individual bits.

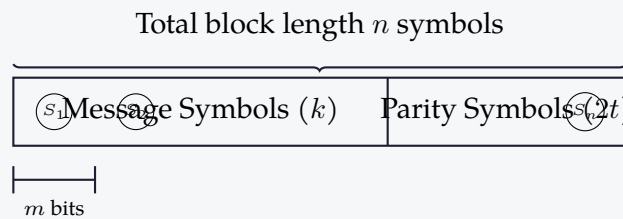
3.1 Basic Parameters

Let m be the number of bits per symbol.

Reed-Solomon Parameters

- **Symbol size:** m bits
- **Block length (n):** $n = 2^m - 1$ symbols (equivalent to $m(2^m - 1)$ bits)
- **Message length (k):** k symbols (equivalent to $m \cdot k$ bits)
- **Parity symbols ($n - k$):** $2t$ symbols
- **Minimum distance (d_{min}):** $d_{min} = n - k + 1$

Reed-Solomon Codeword Structure



3.2 Error and Burst Correction Capability

Since $d_{min} = n - k + 1$, a Reed-Solomon code can correct up to t symbol errors:

$$t = \left\lfloor \frac{n - k}{2} \right\rfloor$$

RS codes are highly effective at correcting **burst errors** (consecutive bit errors). Since a symbol is treated as a single entity, if a burst of errors corrupts multiple bits within a single m -bit symbol, it counts as only **one symbol error**. A t -symbol correcting RS code can easily correct a continuous burst of errors up to $m(t - 1) + 1$ bits.

4. MDS (Maximum Distance Separable) Codes

The Singleton bound states that for any linear block code with parameters $[n, k, d_{min}]$, the minimum distance is bounded by:

$$d_{min} \leq n - k + 1$$

MDS Code Definition

A code is **Maximum Distance Separable (MDS)** if it meets the Singleton bound with strict equality:

$$d_{\min} = n - k + 1$$

4.1 Properties of MDS Codes

MDS codes are optimal because they achieve the maximum possible minimum distance for a given block length and code rate.

- **Generator Matrix Independence:** A code is MDS if and only if **any k columns** of its generator matrix G are linearly independent.
- **Parity Matrix Independence:** A code is MDS if and only if **any $n - k$ columns** of its parity check matrix H are linearly independent.
- **Dual Code:** The dual code of an MDS code is also an MDS code.
- **Examples:** Reed-Solomon codes are the most widely used class of MDS codes.

5. Justeen, Alterant and Goppa Codes**5.1 Justeen Codes**

Justeen codes are a class of algebraic codes constructed by combining Reed-Solomon codes. They are designed to achieve good distance properties with simpler decoding algorithms.

5.2 Alterant Codes

Alterant codes are a generalization of BCH codes. They are defined by selecting roots from an extension field using a set of "locators" and "multipliers" that do not necessarily have the cyclic structure of standard BCH codes. This generalization allows for a wider choice of code lengths.

5.3 Goppa Codes

Goppa codes are an important class of non-cyclic algebraic codes defined by a **Goppa Polynomial** $g(z)$ over a finite field.

Goppa Code Definition

Let $L = \{\gamma_1, \gamma_2, \dots, \gamma_n\}$ be a set of distinct elements in $\text{GF}(2^m)$, and let $g(z)$ be a polynomial over $\text{GF}(2^m)$ such that $g(\gamma_i) \neq 0$ for all i . A Goppa code consists of all vectors $c = (c_1, \dots, c_n)$ that satisfy:

$$\sum_{i=1}^n \frac{c_i}{z - \gamma_i} \equiv 0 \pmod{g(z)}$$

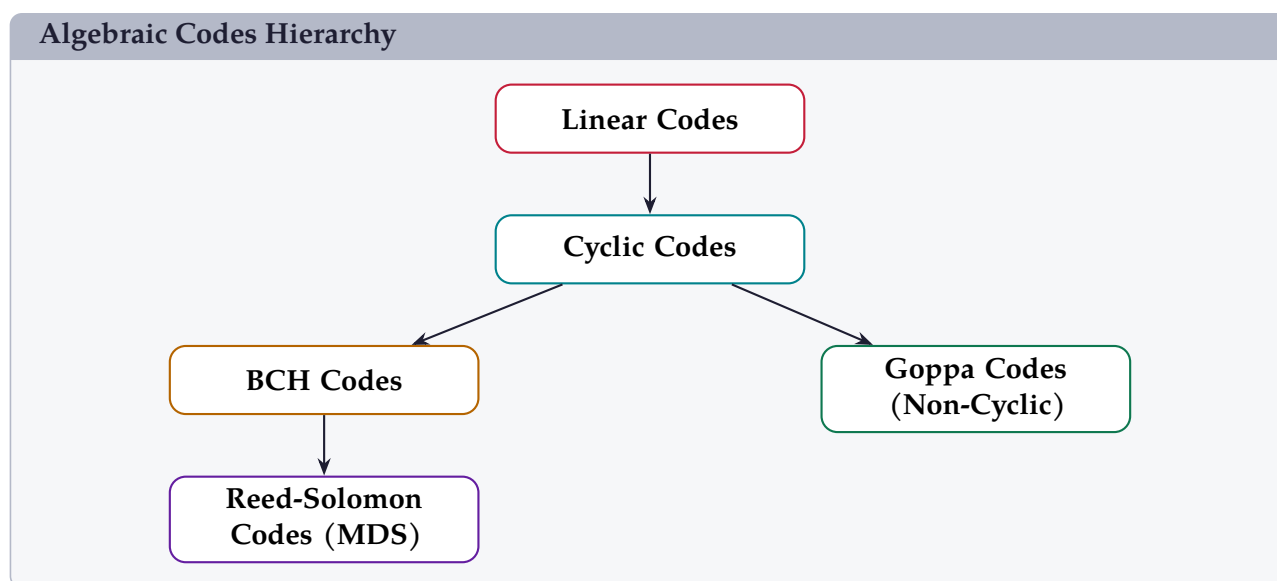
Goppa codes are key to **code-based cryptography** (e.g., the McEliece cryptosystem) because their lack of cyclic structure makes them resistant to algebraic attacks while retaining strong error-correction capabilities.

5.4 Generalized BCH Codes

Generalized BCH codes extend standard BCH codes by relaxing the requirement that the roots be consecutive powers of a primitive element. By choosing an arbitrary set of roots, we can optimize the code for specific channels.

6. Code Classes Hierarchy

The relationship between the various algebraic code classes described can be represented as a hierarchical tree:



Quick Revision — Module 3

BCH, RS and Special Codes | PE-EC801C

Key Formulas and Metrics at a Glance

BCH roots criteria	$g(\alpha^i) = 0$ for $i = 1, 2, \dots, 2t$
BCH bound	$d_{min} \geq 2t + 1$
RS Block length (symbols)	$n = 2^m - 1$
RS Message length (symbols)	$k = n - 2t$
MDS Singleton equality	$d_{min} = n - k + 1$
RS Error correction capability	$t = \lfloor (n - k)/2 \rfloor$ symbol errors
Idempotent condition	$e(x)^2 \equiv e(x) \pmod{X^n \oplus 1}$
Mattson-Solomon polynomial	$g_v(z) = \sum_{j=1}^n V_j z^{n-j}$
Goppa code congruence	$\sum_{i=1}^n \frac{c_i}{z - \gamma_i} \equiv 0 \pmod{g(z)}$

One-Line Definitions

- **BCH Code:** A class of multiple-error-correcting cyclic codes defined by consecutive powers of α as roots.
- **Idempotent Polynomial:** A polynomial that is equal to its own square modulo $X^n \oplus 1$.
- **Reed-Solomon Code:** A non-binary subclass of BCH codes where symbols are from $\text{GF}(2^m)$.
- **Singleton Bound:** The upper limit on the minimum distance of a block code: $d_{min} \leq n - k + 1$.
- **MDS Code:** A code that achieves the Singleton bound with equality.
- **Goppa Code:** A class of non-cyclic algebraic codes defined by a Goppa polynomial $g(z)$ over a finite field.
- **Burst Error:** A sequence of consecutive bit errors caused by channel disturbances.
- **Design Distance (d_d):** The lower bound on minimum distance ($2t + 1$) used to construct a BCH code.

Frequently Asked / Viva Questions

Q1. How is a BCH generator polynomial constructed for a given error capability t ?

We first identify the $2t$ consecutive roots: $\alpha^1, \dots, \alpha^{2t}$. Then, we find the minimal polynomials $m_i(x)$ for each of these roots. The generator polynomial is the least common multiple (LCM) of these minimal polynomials.

Q2. Why are Reed-Solomon codes highly suited for burst error correction?

RS codes operate on m -bit symbols. An error burst of length L bits can corrupt at most $\lceil L/m \rceil + 1$ symbols. Since correcting a symbol corrects all bit errors within it, RS codes handle localized burst corruptions very efficiently.

Q3. What makes a code "Maximum Distance Separable" (MDS)?

A code is MDS if its minimum distance d_{min} reaches the absolute theoretical maximum defined by the Singleton bound: $d_{min} = n - k + 1$. Reed-Solomon codes are the most famous example.

Q4. Explain the role of the Goppa polynomial in cryptography.

Goppa codes are the foundation of McEliece cryptography. Because they lack cyclic structure, their generator matrices appear random to attackers, making it difficult to decode messages without the private Goppa polynomial.

Q5. What is the difference between binary BCH codes and Reed-Solomon codes?

Binary BCH codes process individual bits, and their codewords consist of bits. RS codes are non-binary BCH codes where codewords consist of multi-bit symbols from $GF(2^m)$, offering better performance against symbol-level errors.

Q6. What are idempotents and how do they relate to cyclic codes?

An idempotent is a polynomial satisfying $e(x)^2 \equiv e(x) \pmod{X^n \oplus 1}$. In cyclic codes, the generating idempotent serves as the identity element of the code and uniquely defines the ideal.

Q7. Explain the Singleton bound and its implication.

The Singleton bound states $d_{min} \leq n - k + 1$. It implies that the minimum distance cannot exceed the number of parity check bits plus one, representing the trade-off between redundancy and error capability.

Q8. What are Justesen codes?

Justesen codes are algebraic codes built by combining Reed-Solomon codes, designed to achieve excellent distance properties with simpler, lower-complexity decoders.

Q9. Define the Mattson-Solomon polynomial.

The Mattson-Solomon polynomial represents a vector as a polynomial $g_v(z) = \sum V_j z^{n-j}$ where V_j are its finite field DFT coefficients, mapping time-domain data into the spectral domain.

Q10. Is the actual minimum distance of a BCH code always equal to its design distance?

Frequently Asked / Viva Questions (contd.)

No. The BCH bound guarantees $d_{min} \geq 2t + 1$. While the actual minimum distance is often equal to $2t + 1$, it can sometimes be larger due to the properties of the minimal polynomials.

Comparison of BCH and Reed-Solomon Codes

Property	Binary BCH Codes	Reed-Solomon Codes
Symbol Type	Binary bits (0, 1)	Non-binary symbols from $GF(2^m)$
Block Length (n)	$2^m - 1$ bits	$2^m - 1$ symbols
Minimum Distance	$d_{min} \geq 2t + 1$	$d_{min} = n - k + 1$ (MDS)
Error Target	Random single-bit errors	Burst errors and symbol-level corruptions
Complexity	Moderate	High (requires finite field arithmetic)

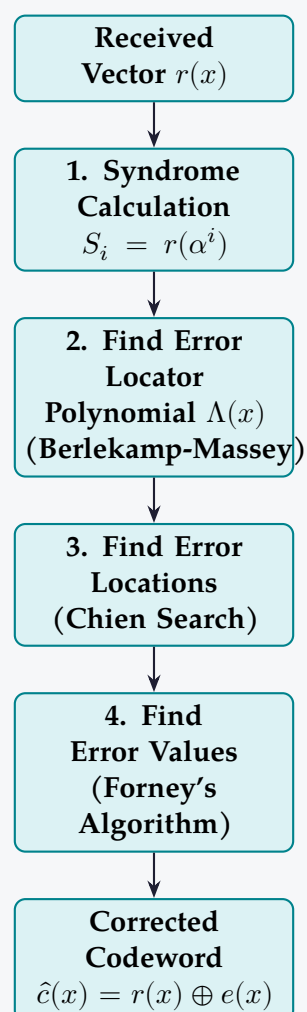
Module 4

The BCH/RS Decoding Pipeline

1. The BCH/RS Decoding Pipeline

Decoding algebraic codes (BCH and Reed-Solomon) is more complex than decoding linear block codes. Instead of simple table lookups, a multi-stage algebraic pipeline computes error locations and error values.

BCH and Reed-Solomon Decoding Flowchart



2. Syndrome Polynomial and the Key Equation

Let $c(x)$ be the transmitted codeword polynomial, $r(x)$ be the received polynomial, and $e(x) = \sum_{j \in \mathcal{E}} Y_j x^j$ be the error polynomial, where $\mathcal{E}$ is the set of error positions and Y_j are the error values ($Y_j \in \text{GF}(2)$ for binary BCH, and $Y_j \in \text{GF}(2^m)$ for RS codes).

2.1 Syndrome Calculations

The decoder calculates $2t$ syndromes:

$$S_i = r(\alpha^i) = c(\alpha^i) \oplus e(\alpha^i) = 0 \oplus e(\alpha^i) = \sum_{l=1}^{\nu} Y_l X_l^i, \quad i = 1, 2, \dots, 2t$$

where ν is the actual number of errors, $X_l = \alpha^{i_l}$ is the **error locator** indicating the position of the l -th error, and Y_l is the error value. We define the **Syndrome Polynomial**:

$$S(x) = \sum_{i=1}^{2t} S_i x^{i-1} = S_1 \oplus S_2 x \oplus S_3 x^2 \oplus \dots \oplus S_{2t} x^{2t-1}$$

2.2 The Error Locator Polynomial

The **Error Locator Polynomial** $\Lambda(x)$ has roots that are the reciprocals of the error locators X_l :

Error Locator Polynomial

$$\Lambda(x) = \prod_{l=1}^{\nu} (1 - X_l x) = 1 + \Lambda_1 x + \Lambda_2 x^2 + \dots + \Lambda_{\nu} x^{\nu}$$

The roots of $\Lambda(x)$ are $x_l = X_l^{-1}$. Finding these roots reveals the error positions.

2.3 The Key Equation

By defining the **Error Evaluator Polynomial** $\Omega(x)$ of degree less than ν , we establish the fundamental relationship of algebraic decoding:

The Key Equation of Decoding

$$\Lambda(x) \cdot S(x) \equiv \Omega(x) \pmod{x^{2t}}$$

The task of the decoder is to solve this equation for $\Lambda(x)$ and $\Omega(x)$, given the known syndrome polynomial $S(x)$.

3. The Berlekamp-Massey Algorithm

The **Berlekamp-Massey (BM)** algorithm is an iterative design procedure that solves the key equation. It views the syndrome values as outputs of a Linear Feedback Shift Register (LFSR) and finds the minimum register length L and feedback connection polynomial $\Lambda^{(r)}(x)$ that can synthesize the sequence $S_1, S_2, \dots, S_{2t}$.

3.1 Iterative Steps of the BM Algorithm

We compute polynomials $\Lambda^{(r)}(x)$ for $r = 1, 2, \dots, 2t$:

Berlekamp-Massey Iteration

At step r , we compute the **discrepancy** d_r :

$$d_r = S_{r+1} \oplus \sum_{i=1}^L \Lambda_i^{(r)} S_{r+1-i}$$

1. If $d_r = 0$, the current LFSR synthesizes the next syndrome:

$$\Lambda^{(r+1)}(x) = \Lambda^{(r)}(x), \quad L^{(r+1)} = L^{(r)}$$

2. If $d_r \neq 0$, the loop coefficients must be adjusted:

$$\Lambda^{(r+1)}(x) = \Lambda^{(r)}(x) - d_r d_p^{-1} x^{r-p} \Lambda^{(p)}(x)$$

where p is a previous step that had a non-zero discrepancy d_p , chosen to minimize the new register length:

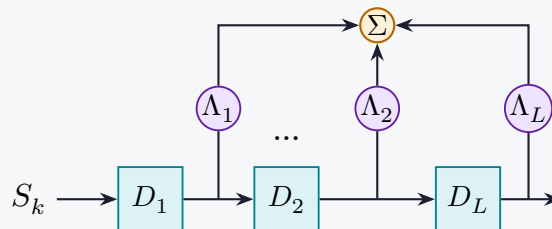
$$L^{(r+1)} = \max(L^{(r)}, r + 1 - L^{(r)})$$

After $2t$ iterations, the resulting polynomial $\Lambda^{(2t)}(x)$ is the desired error locator polynomial $\Lambda(x)$.

4. Massey's Shift Register Synthesis

James Massey showed that the algebraic problem of decoding BCH codes is equivalent to the system-theoretic problem of synthesizing a minimum-length linear feedback shift register that generates a given sequence.

Massey's LFSR Synthesis Model



The feedback connections $\Lambda_1, \dots, \Lambda_L$ represent the coefficients of the error locator polynomial $\Lambda(x) = 1 + \Lambda_1 x + \dots + \Lambda_L x^L$. The discrepancy d_r represents the difference between the actual syndrome S_{r+1} and the output predicted by the shift register.

5. A Fast Berlekamp-Massey Algorithm

To handle high-speed data transmission (e.g., optical networks and solid-state storage), the iterative Berlekamp-Massey algorithm can be reorganized into a parallel, systolic architecture. By utilizing division-free updates and maintaining auxiliary polynomials in parallel, modern decoders compute $\Lambda(x)$ in exactly $2t$ clock cycles without needing finite-field inversions (d_p^{-1}).

6. Chien Search and Forney's Algorithm

Once $\Lambda(x)$ and $\Omega(x)$ are computed, we must identify the error locations and calculate their values.

6.1 Chien Search

Chien search is a hardware-efficient method to find the roots of $\Lambda(x)$. Since the roots must lie in the finite field $\text{GF}(2^m)$, we test all possible elements α^{-i} ($i = 0, \dots, n-1$):

$$\Lambda(\alpha^{-i}) = 1 + \Lambda_1 \alpha^{-i} + \Lambda_2 \alpha^{-2i} + \dots + \Lambda_\nu \alpha^{-\nu i}$$

If $\Lambda(\alpha^{-i}) = 0$, then position i is identified as an error location. In hardware, the terms $\Lambda_j \alpha^{-ji}$ are updated each clock cycle using simple scaling registers, avoiding expensive exponentiations.

6.2 Forney's Algorithm

For non-binary codes (like Reed-Solomon), we must find the error values Y_l to correct the corrupted symbols. Forney's algorithm computes these directly:

Forney's Error Value Formula

The error value Y_l at position i_l (where $X_l = \alpha^{i_l}$) is:

$$Y_l = -\frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})}$$

where $\Lambda'(x)$ is the formal derivative of the error locator polynomial:

$$\Lambda'(x) = \sum_{j=1}^{\nu} j \cdot \Lambda_j x^{j-1}$$

In binary fields, the addition is modulo-2, so the negative sign is ignored, and the derivative $\Lambda'(x)$ simplifies to the sum of its odd-indexed terms.

Quick Revision — Module 4

Decoding of BCH and RS Codes | PE-EC801C

Key Formulas and Metrics at a Glance

Syndrome elements	$S_i = \sum_{l=1}^{\nu} Y_l X_l^i$
Syndrome Polynomial	$S(x) = \sum_{i=1}^{2t} S_i x^{i-1}$
Error Locator Polynomial	$\Lambda(x) = \prod_{l=1}^{\nu} (1 - X_l x)$
Key Equation of Decoding	$\Lambda(x) \cdot S(x) \equiv \Omega(x) \pmod{x^{2t}}$
Discrepancy equation	$d_r = S_{r+1} \oplus \sum_{i=1}^L \Lambda_i^{(r)} S_{r+1-i}$
Register length update	$L^{(r+1)} = \max(L^{(r)}, r+1 - L^{(r)})$
Chien search check	$\Lambda(\alpha^{-i}) = 0 \implies \text{error at index } i$
Forney's error value	$Y_l = -\frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})}$
Formal derivative (binary)	$\Lambda'(x) = \Lambda_1 \oplus \Lambda_3 x^2 \oplus \Lambda_5 x^4 \oplus \dots$

One-Line Definitions

- **Syndrome Polynomial ($S(x)$):** A polynomial constructed from received syndromes representing the error pattern.
- **Error Locator ($\Lambda(x)$):** A polynomial whose roots indicate the positions where transmission errors occurred.
- **Error Evaluator ($\Omega(x)$):** A polynomial containing spectral details used to calculate error magnitudes.
- **Discrepancy (d_r):** The difference between the observed syndrome and the LFSR prediction in BM.
- **Berlekamp-Massey Algorithm:** An iterative, FSR-based procedure used to solve the key equation.
- **Chien Search:** A systematic search method evaluating $\Lambda(x)$ at all field elements to find roots.
- **Forney's Algorithm:** A low-complexity algebraic method used to calculate error values for non-binary codes.
- **Key Equation:** The core modular relationship linking $\Lambda(x)$, $S(x)$, and $\Omega(x)$ in algebraic decoding.

Frequently Asked / Viva Questions

Q1. What are the four main steps of the BCH/RS decoding pipeline?

First, compute the syndrome values from the received vector. Second, apply the Berlekamp-Massey algorithm to compute the error locator polynomial $\Lambda(x)$. Third, perform Chien search to find the roots of $\Lambda(x)$, locating the errors. Fourth, use Forney's algorithm to compute error values.

Q2. State the Key Equation of decoding and define its terms.

The Key Equation is $\Lambda(x) \cdot S(x) \equiv \Omega(x) \pmod{x^{2t}}$, where $\Lambda(x)$ is the error locator polynomial, $S(x)$ is the syndrome polynomial, and $\Omega(x)$ is the error evaluator polynomial. It links the time and spectral representations of errors.

Q3. How does the Berlekamp-Massey algorithm model the decoding problem?

The BM algorithm models the syndrome sequence as outputs of a Linear Feedback Shift Register (LFSR). It iteratively constructs the shortest LFSR that generates the syndromes, which represents the minimum-degree error locator.

Q4. Explain the concept of 'discrepancy' in the BM algorithm.

Discrepancy (d_r) is the difference between the actual syndrome S_{r+1} and the value predicted by the feedback connections of the current iteration. If $d_r = 0$, the connections are correct; if $d_r \neq 0$, the connection polynomial must be adjusted.

Q5. What is Chien search, and how does it optimize hardware implementation?

Chien search finds the roots of $\Lambda(x)$ by evaluating it at all field elements α^{-i} . In hardware, terms are updated using multipliers that scale by constant values (powers of α), replacing complex exponentiations with simple shifts.

Q6. Explain Forney's algorithm and why it is useful.

Forney's algorithm calculates error values using $Y_i = -\Omega(X_i^{-1})/\Lambda'(X_i^{-1})$. It is highly useful because it allows direct computation of error values at known error locations without solving a large system of simultaneous equations.

Q7. Why does the formal derivative of a polynomial simplify in binary fields?

In binary fields, addition is modulo-2. Since $2 \equiv 0 \pmod{2}$, all terms with even powers in the derivative disappear (since their derivatives contain an even multiplier: $2 \cdot c \cdot x = 0$). Thus, only the odd-indexed terms remain.

Q8. Can the BM algorithm handle erasure decoding?

Yes. Erasures are errors with known locations but unknown values. The BM algorithm can be initialized with an erasure locator polynomial, reducing the number of syndrome iterations needed to solve for the remaining unknown errors.

Q9. What is the difference between Berlekamp-Massey and the Euclidean decoding algorithm?

BM is an iterative LFSR design process that runs in $2t$ steps. The Euclidean algorithm solves the key equation by finding the greatest common divisor of x^{2t} and $S(x)$ using polynomial division. Both produce identical results.

Frequently Asked / Viva Questions (contd.)

Q10. What happens during decoding if the number of channel errors exceeds t ?

If the number of errors exceeds t , the Berlekamp-Massey algorithm will still compute a polynomial, but it will have invalid roots, or the roots won't correspond to correct locations, resulting in a decoding failure or undetected error.

Comparison of Decoding Algorithms

Property	Berlekamp-Massey Algorithm	Euclidean Algorithm
Mathematical Basis	Linear feedback shift register synthesis	Polynomial GCD division
Execution Flow	Iterative (updates loop coefficients)	Recursive division steps
Hardware Suitability	Excellent (systolic array architectures)	Challenging (requires polynomial dividers)
Design Focus	Solving the LFSR feedback connections	Direct fraction reduction of $S(x)/x^{2t}$

Module 5

Introduction to Convolutional Codes

1. Introduction to Convolutional Codes

Unlike linear block codes, which segment the incoming message sequence into independent blocks of length k , **convolutional codes** process the message continuously. The coded output depends not only on the current input block but also on a finite number of previous input blocks stored in memory.

1.1 Core Parameters

A convolutional encoder is characterized by three main parameters:

1. **Code Rate (R):** Defined as $R = k/n$, where k input bits produce n output bits in each encoding cycle.
2. **Memory Stages (M):** The number of delay elements (flip-flops) in the shift registers.
3. **Constraint Length (K):** There are two standard definitions:
 - **System-Theoretic definition:** $K = M + 1$. This represents the maximum number of input cycles over which a single input bit can influence the encoder's output.
 - **Alternative definition:** $K = L \cdot k$, where L is the number of stages in the shift register.

1.2 Mathematical Representations

The encoding operation can be described in three algebraic formats:

Mathematical Formulations of Convolutional Coding

1. **Impulse Response:** Let $h^{(j)} = (h_0^{(j)}, h_1^{(j)}, \dots)$ be the impulse response of the path from the input to output j . For input sequence $u = (u_0, u_1, \dots)$, the output $v^{(j)}$ is the discrete convolution:

$$v_t^{(j)} = \sum_{i=0}^M u_{t-i} h_i^{(j)}$$

2. **Generator Matrix (G):** A semi-infinite, Block-Toeplitz matrix of the form:

$$G = \begin{bmatrix} G_0 & G_1 & G_2 & \dots & G_M & 0 & \dots \\ 0 & G_0 & G_1 & \dots & G_{M-1} & G_M & \dots \\ 0 & 0 & G_0 & \dots & G_{M-2} & G_{M-1} & \dots \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots \end{bmatrix}$$

where each G_i is a $k \times n$ sub-matrix representing tap connections.

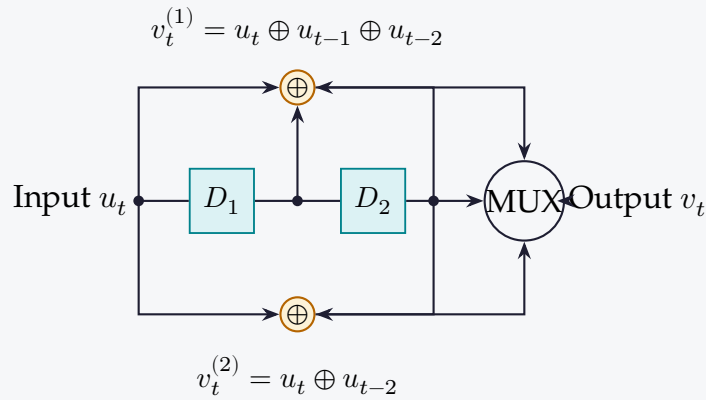
3. **Generator Polynomials:** Tap connections for the j -th output are expressed as a polynomial in the delay operator D :

$$g^{(j)}(D) = g_0^{(j)} \oplus g_1^{(j)} D \oplus g_2^{(j)} D^2 \oplus \dots \oplus g_M^{(j)} D^M$$

2. Structural Representations

Convolutional encoders are implemented using feed-forward shift registers and modulo-2 adders. They can be analyzed using three graphical models.

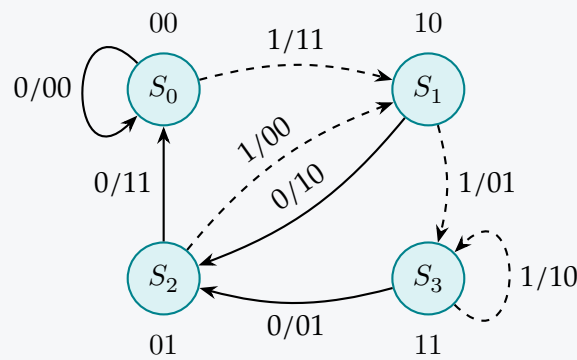
Feed-Forward Encoder for a Rate 1/2 ($K = 3$) Convolutional Code



2.1 State Transition Diagram

A state transition diagram represents the encoder as a finite-state machine. The state is determined by the previous bits stored in the shift register: $S_t = (u_{t-1}, u_{t-2}, \dots, u_{t-M})$. For $M = 2$, there are four possible states: 00, 10, 01, and 11.

State Transition Diagram (4-State, K=3 Encoder)

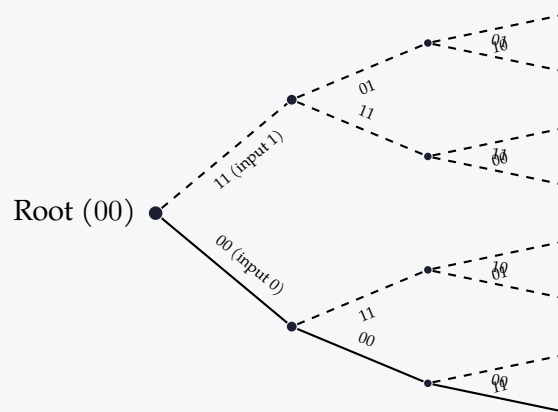


Legend: Solid lines represent transitions caused by input '0'. Dashed lines represent transitions caused by input '1'. Labels indicate Input/Output.

2.2 Tree Diagram

The tree diagram represents all possible output paths starting from the initial state 00. For each incoming bit, the path splits into two branches: the upper branch represents input 0, and the lower branch represents input 1. The diagram grows exponentially, making it suitable only for illustrating short sequences.

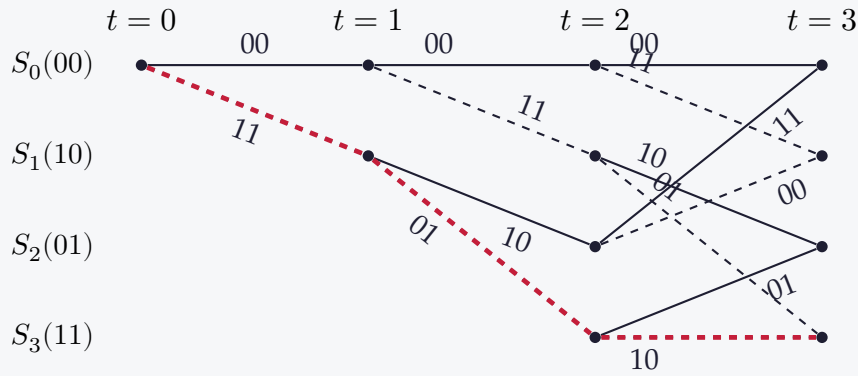
3-Level Tree Diagram Representation



2.3 Trellis Diagram

By merging node branches corresponding to identical states in the tree diagram, we obtain the **trellis diagram**. The trellis diagram is a time-indexed state diagram. Since the number of states is fixed (2^M), the trellis width is constant for all time intervals $t \geq M$, providing a highly compact layout for analyzing long sequences.

Trellis Diagram Section showing State Transitions and Survivor Path



3. Distance Properties of Convolutional Codes

The error-correcting capability of a convolutional code depends on its distance properties.

3.1 Free Distance (d_{free})

The most important distance metric is the **free distance** d_{free} .

Free Distance Definition

The free distance d_{free} is the minimum Hamming distance between any two distinct code sequences in the trellis that begin at state 00 and eventually diverge, only to merge back at state 00 at some later step.

$$d_{\text{free}} = \min \{d_H(v^{(1)}, v^{(2)}) \mid u^{(1)} \neq u^{(2)}\}$$

If we set one sequence to be the all-zero codeword, d_{free} simplifies to the minimum weight of a non-zero code sequence starting and ending at state 00. The number of errors t that the code can correct is:

$$t = \left\lfloor \frac{d_{\text{free}} - 1}{2} \right\rfloor$$

3.2 Catastrophic Error Propagation

Catastrophic error propagation is a key hazard in convolutional code design.

Catastrophic Error Propagation

A code is **catastrophic** if a finite number of channel transmission errors leads to an infinite number of decoding decisions/bit errors in the reconstructed message sequence.

Sufficient Condition: In the state transition diagram of a catastrophic code, there exists a closed loop (cycle) containing a state other than 00 where all transitions along the loop produce zero-weight output bits (all output bits are '0'), despite being triggered by non-zero inputs. For systematic convolutional codes, catastrophic propagation is impossible because the inputs are mirrored directly in the output stream, preventing zero-weight outputs for non-zero inputs.

4. The Viterbi Decoding Algorithm

The **Viterbi algorithm** is a Maximum Likelihood (ML) decoding technique that minimizes the probability of sequence error. It systematically searches the code trellis to find the path that is closest to the received vector.

4.1 Algorithmic Steps

Let the received sequence be $r = (r_0, r_1, \dots, r_{L-1})$. At each time step t , the decoder maintains a single survivor path for each state, along with its accumulated path metric:

1. **Branch Metric (BM):** Compute the distance between the received segment r_t and the expected output on each branch:
 - **Hard Decision:** Hamming distance $d_H(r_t, v_{\text{branch}})$.
 - **Soft Decision:** Euclidean distance $\|r_t - v_{\text{branch}}\|^2$.
2. **Path Metric (PM):** Calculate candidates for each state:

$$PM_t(S) = \min_{S' \rightarrow S} [PM_{t-1}(S') + BM(S' \rightarrow S)]$$

3. **Add-Compare-Select (ACS):** For each state S at step t :
 - **Add** branch metrics to the previous path metrics of all incoming states S' .
 - **Compare** the candidate path metrics.
 - **Select** the path with the minimum metric as the survivor. Discard all other incoming paths.
4. **Traceback:** Once the end of the trellis is reached (or after a fixed decoding depth), trace the path backward from the state with the lowest path metric to recover the input sequence.

5. Sequential Decoding Algorithms

Sequential decoding algorithms are sub-optimal, tree-based decoding techniques. Unlike Viterbi, which searches all trellis paths in parallel, sequential decoders explore only one path at a time. They are suitable for codes with large constraint lengths K , where Viterbi's complexity (2^{K-1}) is prohibitive.

5.1 Wozencraft's Algorithm

The pioneer sequential decoding algorithm, Wozencraft's algorithm, performs a depth-first search on the code tree. It traces a path forward. If the accumulated path metric exceeds a threshold (indicating too many errors), the decoder halts, backs up (backtracks), and attempts to explore alternative branches.

5.2 Fano's Algorithm (Fann's Algorithm)

Fano's algorithm (referred to as Fann's in some syllabi) is a refinement of tree search that operates without a storage stack:

- It uses a variable threshold that is dynamically adjusted based on channel quality.
- The decoder moves forward along the branch that maximizes the Fano metric (a metric that penalizes paths that deviate from typical channel characteristics).
- If the path metric falls below the threshold, the decoder lowers the threshold and backs up to explore other paths.
- **Advantage:** Low memory requirements. **Disadvantage:** Variable decoding time depending on channel noise.

5.3 The Stack Algorithm (Zigangirov-Jelinek)

The Stack (or priority queue) algorithm is a fast sequential search method:

- The decoder maintains a list (stack) of explored paths of varying lengths, ordered by their path metrics.
- In each cycle, the decoder removes the path with the best metric from the top of the stack.
- It extends this path by computing metrics for its two child branches and inserts the new paths back into the stack.
- The stack is re-sorted, and the process repeats until a path reaches the end of the tree.
- **Advantage:** Eliminates backtracking. **Disadvantage:** Requires a large memory space to maintain and sort the stack.

6. Worked Example of Viterbi Decoding

Consider the $(2, 1, 2)$ convolutional code shown in Section 2. The encoder starts at state $S_0(00)$. We receive the sequence $r = (11, 01, 01, 00)$. Let's decode it using the Viterbi algorithm.

6.1 Step-by-Step Trellis Computations

Step 1: Time $t = 1$ (Received $r_0 = 11$)

The only active starting state is $S_0(00)$ with path metric $PM_0(S_0) = 0$.

- Transition $S_0 \rightarrow S_0$ (input 0): output 00. $BM = d_H(11, 00) = 2$. $PM_1(S_0) = PM_0(S_0) + 2 = 2$. (Survivor: $S_0 \rightarrow S_0$)
- Transition $S_0 \rightarrow S_1$ (input 1): output 11. $BM = d_H(11, 11) = 0$. $PM_1(S_1) = PM_0(S_0) + 0 = 0$. (Survivor: $S_0 \rightarrow S_1$)

States S_2 and S_3 are unreachable at $t = 1$ ($PM_1 = \infty$).

Step 2: Time $t = 2$ (Received $r_1 = 01$)

We transition from active states S_0 ($PM_1 = 2$) and S_1 ($PM_1 = 0$):

- From S_0 :
 - To S_0 (input 0, output 00): $BM = d_H(01, 00) = 1$. Candidate $PM = 2 + 1 = 3$.
 - To S_1 (input 1, output 11): $BM = d_H(01, 11) = 1$. Candidate $PM = 2 + 1 = 3$.
- From S_1 :
 - To S_2 (input 0, output 10): $BM = d_H(01, 10) = 2$. Candidate $PM = 0 + 2 = 2$.
 - To S_3 (input 1, output 01): $BM = d_H(01, 01) = 0$. Candidate $PM = 0 + 0 = 0$.

Since there are no collisions yet, the survivors at $t = 2$ are: $PM_2(S_0) = 3$, $PM_2(S_1) = 3$, $PM_2(S_2) = 2$, and $PM_2(S_3) = 0$.

Step 3: Time $t = 3$ (Received $r_2 = 01$)

All states are active. We apply the Add-Compare-Select (ACS) operation for each state:

- **State $S_0(00)$:**
 - From S_0 ($PM = 3$): branch output 00, $BM = d_H(01, 00) = 1$. Path candidate = $3 + 1 = 4$.
 - From S_2 ($PM = 2$): branch output 11, $BM = d_H(01, 11) = 1$. Path candidate = $2 + 1 = 3$.
 - **Select:** $PM_3(S_0) = \min(4, 3) = 3$. (Survivor path came from S_2).
- **State $S_1(10)$:**
 - From S_0 ($PM = 3$): branch output 11, $BM = d_H(01, 11) = 1$. Path candidate = $3 + 1 = 4$.
 - From S_2 ($PM = 2$): branch output 00, $BM = d_H(01, 00) = 1$. Path candidate = $2 + 1 = 3$.
 - **Select:** $PM_3(S_1) = \min(4, 3) = 3$. (Survivor path came from S_2).
- **State $S_2(01)$:**
 - From S_1 ($PM = 3$): branch output 10, $BM = d_H(01, 10) = 2$. Path candidate = $3 + 2 = 5$.
 - From S_3 ($PM = 0$): branch output 01, $BM = d_H(01, 01) = 0$. Path candidate = $0 + 0 = 0$.
 - **Select:** $PM_3(S_2) = \min(5, 0) = 0$. (Survivor path came from S_3).
- **State $S_3(11)$:**
 - From S_1 ($PM = 3$): branch output 01, $BM = d_H(01, 01) = 0$. Path candidate = $3 + 0 = 3$.
 - From S_3 ($PM = 0$): branch output 10, $BM = d_H(01, 10) = 2$. Path candidate = $0 + 2 = 2$.
 - **Select:** $PM_3(S_3) = \min(3, 2) = 2$. (Survivor path came from S_3).

Step 4: Time $t = 4$ (Received $r_3 = 00$)

We perform ACS for the final time step:

- **State $S_0(00)$:**

- From S_0 ($PM_3 = 3$): branch output 00, $BM = d_H(00, 00) = 0$. Path candidate = $3 + 0 = 3$.
- From S_2 ($PM_3 = 0$): branch output 11, $BM = d_H(00, 11) = 2$. Path candidate = $0 + 2 = 2$.
- **Select:** $PM_4(S_0) = \min(3, 2) = 2$. (Survivor path came from S_2).
- **State $S_1(10)$:**
 - From S_0 ($PM_3 = 3$): branch output 11, $BM = d_H(00, 11) = 2$. Path candidate = $3 + 2 = 5$.
 - From S_2 ($PM_3 = 0$): branch output 00, $BM = d_H(00, 00) = 0$. Path candidate = $0 + 0 = 0$.
 - **Select:** $PM_4(S_1) = \min(5, 0) = 0$. (Survivor path came from S_2).
- **State $S_2(01)$:**
 - From S_1 ($PM_3 = 3$): branch output 10, $BM = d_H(00, 10) = 1$. Path candidate = $3 + 1 = 4$.
 - From S_3 ($PM_3 = 2$): branch output 01, $BM = d_H(00, 01) = 1$. Path candidate = $2 + 1 = 3$.
 - **Select:** $PM_4(S_2) = \min(4, 3) = 3$. (Survivor path came from S_3).
- **State $S_3(11)$:**
 - From S_1 ($PM_3 = 3$): branch output 01, $BM = d_H(00, 01) = 1$. Path candidate = $3 + 1 = 4$.
 - From S_3 ($PM_3 = 2$): branch output 10, $BM = d_H(00, 10) = 1$. Path candidate = $2 + 1 = 3$.
 - **Select:** $PM_4(S_3) = \min(4, 3) = 3$. (Survivor path came from S_3).

6.2 Traceback Process

At the end of the trellis ($t = 4$), the state with the minimum path metric is S_1 with $PM_4(S_1) = 0$. We trace this path backward:

1. At $t = 4$, we are in state $S_1(10)$. The survivor came from state $S_2(01)$ at $t = 3$. The transition $S_2(01) \rightarrow S_1(10)$ is triggered by input bit **1**.
2. At $t = 3$, we are in state $S_2(01)$. The survivor came from state $S_3(11)$ at $t = 2$. The transition $S_3(11) \rightarrow S_2(01)$ is triggered by input bit **0**.
3. At $t = 2$, we are in state $S_3(11)$. The survivor came from state $S_1(10)$ at $t = 1$. The transition $S_1(10) \rightarrow S_3(11)$ is triggered by input bit **1**.
4. At $t = 1$, we are in state $S_1(10)$. The survivor came from state $S_0(00)$ at $t = 0$. The transition $S_0(00) \rightarrow S_1(10)$ is triggered by input bit **1**.

Thus, the decoded message sequence is $u = (1, 1, 0, 1)$, and the transmitted codeword is $c = (11, 01, 01, 00)$. Since $r = c$, the accumulated Hamming distance is indeed 0.

Quick Revision — Module 5

Convolutional Codes and Decoding | PE-EC801C

Key Formulas and Metrics at a Glance

Code Rate	$R = \frac{k}{n}$
Constraint Length (Standard)	$K = M + 1$ (M is the number of delay stages)
Impulse Response Convolution	$v_t^{(j)} = \sum_{i=0}^M u_{t-i} h_i^{(j)}$
Discrete Delay Polynomial	$g^{(j)}(D) = g_0^{(j)} \oplus g_1^{(j)} D \oplus \dots \oplus g_M^{(j)} D^M$
Error Correction Power	$t = \left\lfloor \frac{d_{\text{free}} - 1}{2} \right\rfloor$
Viterbi Path Metric Update	$PM_t(S) = \min_{S' \rightarrow S} [PM_{t-1}(S') + BM(S' \rightarrow S)]$

One-Line Definitions

- **Convolutional Code:** A continuous coding technique where outputs are generated by convolving current and past message bits.
- **Constraint Length (K):** The number of input bits that reside in the encoder and influence the output.
- **Trellis Diagram:** A time-indexed state diagram showing all possible transitions over time.
- **Free Distance (d_{free}):** The minimum Hamming distance between any two distinct paths in the trellis starting and ending in 00.
- **Catastrophic Code:** A code where a finite number of channel errors causes an infinite number of decoded bit errors.
- **Survivor Path:** The path entering a trellis state that has the minimum accumulated path metric.
- **ACS (Add-Compare-Select):** The core hardware operation in Viterbi decoding that updates path metrics and retains survivors.
- **Fano Metric:** A dynamic search metric that penalizes paths that deviate from typical channel behavior.

Frequently Asked / Viva Questions

Q1. What is the difference between block codes and convolutional codes?

Linear block codes process messages in independent block segments, and the current output block depends only on the current input block. Convolutional codes process the input continuously, and the output depends on the current input and a finite memory of past inputs.

Q2. Define constraint length K in a convolutional encoder.

Constraint length K represents the number of input cycles over which an input bit remains in the shift register and influences the output. It is mathematically defined as $K = M + 1$, where M is the number of memory delay elements.

Q3. What is free distance d_{free} and why is it important?

Free distance d_{free} is the minimum Hamming distance between any two paths in the trellis that diverge from the all-zero state and merge back later. It determines the code's error-correcting capability: $t = \lfloor (d_{\text{free}} - 1)/2 \rfloor$.

Q4. How do you identify a catastrophic convolutional code from its state diagram?

A code is catastrophic if its state transition diagram contains a loop of non-zero states where all outputs are zero. If transmission errors cause the decoder to enter this loop, it will continue to output errors indefinitely.

Q5. State the three structural representations of convolutional codes.

The three structural representations are the State Transition Diagram (FSM representation), the Tree Diagram (exponential tree branching showing all output sequences), and the Trellis Diagram (compact time-aligned state transitions).

Q6. Explain the ACS operation in the Viterbi algorithm.

ACS stands for Add-Compare-Select. For each trellis state, it adds the branch metrics of incoming branches to the path metrics of their origin states, compares the candidate values, and selects the path with the minimum metric as the survivor.

Q7. What is the difference between hard-decision and soft-decision decoding?

Hard-decision decoding processes demodulated binary data ('0' or '1') and uses Hamming distance. Soft-decision decoding processes unquantized analog voltage values directly from the channel and uses Euclidean distance, offering a 2–3 dB coding gain.

Q8. Compare Viterbi decoding with sequential decoding.

Viterbi decoding is a maximum likelihood algorithm that searches all trellis paths in parallel, requiring 2^M state processors. Sequential decoding explores a single path at a time in a tree structure, using backtracking or stacks, which reduces memory but introduces variable decoding delays.

Q9. How does Fano's sequential decoding algorithm work?

Frequently Asked / Viva Questions (contd.)

Fano's algorithm performs a tree search by moving forward or backward along branches based on a metric threshold. If the path metric falls below the threshold, it lowers the threshold and backtracks, requiring no memory stacks.

Q10. What is the Stack algorithm, and what is its main drawback?

The Stack algorithm (Zigangirov-Jelinek) maintains a priority queue of paths ordered by their metrics, extending the top path. Its main drawback is the large memory required to store and sort the paths, and the risk of stack overflow under noisy conditions.

Structural Comparison of Block and Convolutional Codes

Property	Linear Block Codes	Convolutional Codes
Encoding Mode	Block-by-block (independent segments)	Continuous stream processing
Memory Structure	Memoryless (outputs depend on current block)	Finite memory (outputs convolved with history)
Key Parameters	Code length n , message length k	Rate k/n , constraint length K , memory M
Decoding Algorithm	Syndromes table lookup / Meggitt decoding	Viterbi / Sequential decoding

Comparison of Viterbi vs. Sequential Decoding

Property	Viterbi Decoding	Sequential Decoding
Search Strategy	Parallel search over all trellis paths	Single-path depth-first tree search
Optimality	Maximum Likelihood (optimal)	Sub-optimal
Decoding Time	Constant (independent of channel noise)	Variable (increases with channel noise)
Constraint Limit	Limited to small K ($K \leq 9$)	Can decode large K ($K \geq 20$)
Memory Demand	Low storage (only survivor paths)	High storage (stack) or backtracking overhead